UNIVERSIDADE FEDERAL
DE SANTA CATARINA

Nomes: Elisa Belquis e Thayssa Godoi
Matrículas: 24202220 e 19201619
Disciplina: Organização de Computadores I
Professor: Marcelo Berejuk

LABORATÓRIO 7 - MATRIZ EM CACHE

1. INTRODUÇÃO

O presente relatório utiliza os programas desenvolvidos na atividade de laboratório Matrizes em Memória, na qual foi preciso implementar dois algoritmos diferentes de preenchimento de matrizes. Ambos percorrem uma matriz inteira de 16 por 16 elementos, atribuindo aos elementos os valores de 0 a 255 e depois imprimem a mesma. O primeiro programa realiza tal operação percorrendo a matriz **linha por linha**, enquanto o segundo atribuí os elementos **coluna por coluna**. Nos dois casos, a matriz ocupa um espaço total de 16 x 16 x 4 = 1024 bytes.

Utilizando a ferramenta *Data Cache Simulator* do MARS, pudemos avaliar o desempenho de ambas as implementações para diferentes organizações de cache. Considerou-se as políticas de alocação na cache Direct Mapping, Fully Associative N-Way Set Associative.

O simulador mostra apenas o valor de *Cache Hit Rate*. Então, considerando que se não foi um hit, é porque aconteceu um miss, utilizamos os valores de contagem e calculamos a taxa de acerto da memória cache. Os resultados de *Cache Miss Rate* serão apresentados com uma tabela e discutidos para cada um dos programas.

Faz-se importante ressaltar que o acesso à memória de dados acontece devido a duas categorias principais: os acessos à matriz (tanto para escrita no processo de preenchimento, como para leitura na impressão da matriz) e os acessos às strings de I/O (leituras das strings de texto). O "Memory Access Count" do simulador contabiliza todos os acessos a *.data*, e não apenas os acessos diretos à matriz. Como em cada implementação imprime strings diferentes para apresentar a matriz, o número de acessos de cada um dos programas também difere.

1.1. POLÍTICAS DE ALOCAÇÃO

A Política de Alocação (ou Placement Policy) determina onde um bloco de dados da memória principal pode ser colocado dentro da cache. Diferentes políticas de associação geram custos de hardware e eficiência diversos para um mesmo programa.

1.1.1. Mapeamento Direto:

É a política mais simples e restritiva. A definição de em qual linha da cache (*L*) ficará o bloco da memória principal com *x* palavras (neste cálculo não importa a quantidade de palavras) se dá pelo resto da divisão (módulo) do número do bloco em que está a informação desejada (*B*) pelo número total de linhas da cache (N_{TL}).

$$L = B \% N_{TL}$$

Apresenta a vantagem de ser o hardware mais barato e simples de implementar dentre as políticas de associação. Além disso, a verificação de hit/miss é rápida. O controlador só precisa verificar uma única etiqueta (tag) na posição calculada, mas é altamente suscetível a misses de conflito. Se houver o azar de dois blocos de memória que são usados frequentemente serem mapeados para a mesma posição na cache, eles irão "expulsar" um ao outro repetidamente, mesmo que o resto da memória cache esteja vazio.

1.1.2. Mapeamento Totalmente Associativo:

Os blocos da memória principal são colocados inicialmente de forma sequencial nas linhas da cache. A partir do momento em que a cache está cheia e precisa-se substituir alguma de suas linhas, é utilizado um algoritmo de substituição de linha de cache para decidir em qual das linhas previamente ocupadas o bloco será alocado, por exemplo o LRU (Least Recently Used).

É a política que, teoricamente, gera o menor número de misses de conflito. Um bloco só é expulso se a cache estiver totalmente cheia, e mesmo assim, a escolha é feita de forma inteligente. Entretanto, é o hardware mais caro e complexo e lento, pois para verificar um hit/miss, o controlador precisa comparar a etiqueta do endereço desejado com todas as etiquetas presentes na cache simultaneamente.

1.1.3. Mapeamento Associativo por Conjunto de N Sets:

A cache é dividida em Conjuntos (Sets). Um bloco da memória principal mapeia para exatamente um conjunto, mas pode ser colocado em qualquer via (Way) dentro daquele conjunto.

O mapeamento para qual dos conjuntos o bloco vai ser alocado depende do mesmo cálculo realizado na política de mapeamento direto. Entretanto, neste caso, utilizamos o número total de sets (N_{TS}) ao invés do número total de linhas da cache.

$$Set = B \% N_{TS}$$

Um detalhe importante é que se for 1-Way Set Associative (N=1), é a exata mesma coisa que o mapeamento direto. Já se for M-Way Set Associative (onde M é o número total de linhas da cache), é totalmente associativa. Quando bem dimensionada, ela oferece o equilíbrio de uma redução drástica nos misses de conflito (comparado ao direct mapping) sem o alto custo da política fully associative.

2. RESULTADOS

2.1. TABELAS DE DADOS

2.1.1. Linha por Linha

O exercício 1 do laboratório Matrizes em Memória solicitava que fosse implementado o seguinte algoritmo em asssembly:

```
for (row = 0; row < 16; row++)
  for (col = 0; col < 16; col++)
    data[row][col] = value++;
```

Para cada caso, ocorreram 1609 acessos à memória.

Placement Policy	**Blocos**	**Words**	**Cache Miss Count**	**Cache Miss Rate**	**Cache Hit Rate**
Direct Mapping	8	4	188	11,68%	88,32%
		8	127	7,93%	92,07%
	16	8	97	6,03%	93,97%
		16	48	2,98%	97,02%
		32	9	0,56%	99,44%
Fully Associative	8	4	132	8,20%	91,80%
		8	67	4,16%	95,84%
	16	8	67	4,16%	95,84%
		16	33	2,05%	97,95%
		32	9	0,56%	99,44%
N-way Set Associative	8	4	188	11,68%	88,32%

		8	127	7,93%	92,07%
	16	8	97	6,03%	93,97%
		16	48	2,98%	97,02%
		32	9	0,56%	99,44%

2.1.2. Coluna por coluna

O exercício 2 solicitava que fosse implementado o seguinte algoritmo:

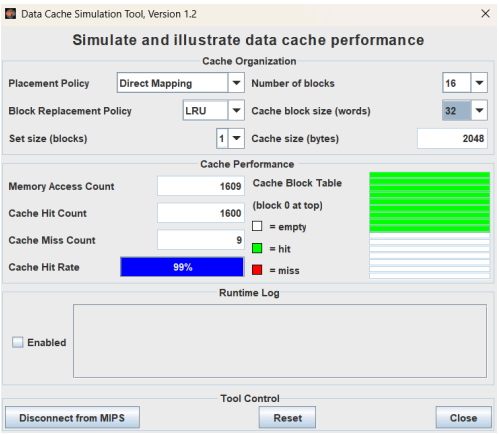
```
for (col = 0; col < 16; col ++)
  for (row = 0; row < 16; row++)
    data[row][col] = value++;
```

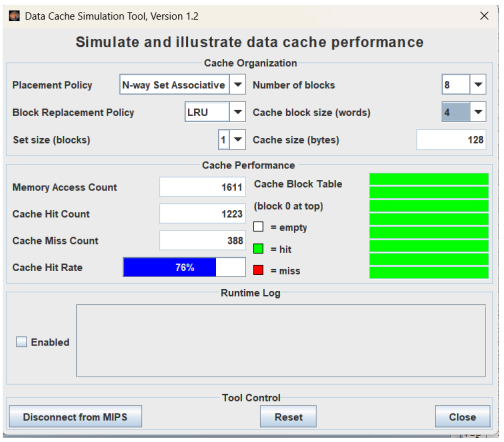
Para cada caso ocorreram 1611 acessos à memória.

Placement Policy	**Blocos**	**Words**	**Cache Miss Count**	**Cache Miss Rate**	**Cache Hit Rate**
Direct Mapping	8	4	388	24,08%	75,92%
		8	351	21,78%	78,22%
	16	8	321	19,93%	80,07%
		16	48	2,98%	97,02%
		32	9	0,56%	99,44%
Fully Associative	8	4	325	20,17%	79,83%
		8	291	18,06%	81,94%
	16	8	67	4,16%	95,84%
		16	33	2,05%	97,95%
		32	9	0,56%	99,44%
N-way Set Associative	8	4	388	24,08%	75,92%
		8	351	21,78%	78,22%
	16	8	321	19,93%	80,07%
		16	48	2,98%	97,02%
		32	9	0,56%	99,44%

2.2. DISCUSSÃO DOS RESULTADOS

Simulações realizadas para linha por linha e coluna por coluna respectivamente





A análise dos dados obtidos para os dois algoritmos demonstra o impacto fundamental da localidade espacial no desempenho da memória cache. O programa "Linha por Linha", que acessa a memória sequencialmente (alta localidade), serve como base de desempenho ideal. Nele, a contagem de misses reduz consistentemente com o aumento do tamanho do bloco. Por exemplo, no Mapeamento Direto com 8 blocos, dobrar o tamanho do bloco de 4 para 8 words reduz os misses de 188 para 127, pois um bloco maior satisfaz mais acessos futuros.

Em contraste, o algoritmo "Coluna por Coluna" exibe um padrão de acesso hostil (baixa localidade), "saltando" 64 bytes (16 palavras) entre acessos de escrita. O impacto é severo: no mesmo cenário de 8 blocos e 4 words, o Mapeamento Direto sofre 388 misses (24,08%), um número 106% maior que os 188 misses do "Linha por Linha". Isso demonstra um clássico cache thrashing, onde o stride do acesso força blocos a serem mutuamente expulsos da cache.

O benefício da associatividade plena (Fully Associative) torna-se evidente nesta comparação. Para o "Linha por Linha", a política Totalmente Associativa oferece uma melhoria moderada, reduzindo misses de conflito pontuais (ex: de 48 para 33 misses no cenário de 16 blocos/16 words). No entanto, para o "Coluna por Coluna", a melhoria é drástica: no cenário de 16 blocos e 8 words, os misses despencam de 321 (Mapeamento Direto) para 67 (Totalmente Associativa). Notavelmente, este valor (67 misses) é idêntico ao do algoritmo "Linha por Linha" na mesma configuração, provando que a política Fully Associative neutralizou

completamente a penalidade do padrão de acesso de baixa localidade, eliminando os misses de conflito sistemáticos.

Finalmente, dois pontos de convergência são idênticos para ambos os algoritmos. Primeiro, os dados da política N-way Set Associative são idênticos aos do Direct Mapping, pois acabamos executando a simulação como 1-Way, que é funcionalmente equivalente ao Mapeamento Direto. Segundo, no cenário final (16 blocos, 32 words), onde a cache de 2048 bytes excede o working set (matriz de 1024 bytes + strings), todas as políticas de ambos os algoritmos convergem para exatamente 9 misses (0,56%). Isso confirma que os misses de capacidade e conflito foram eliminados, e este valor representa o número mínimo e inevitável de misses compulsórios para carregar os dados na cache pela primeira vez.

3. CONCLUSÃO

Este laboratório validou empiricamente a interação fundamental entre algoritmos de software e a arquitetura de hardware da cache. A análise demonstrou que o algoritmo "Coluna por Coluna" (baixa localidade espacial) sofre penalidades severas em políticas restritivas, causando cache thrashing e resultando em 388 misses no Direct Mapping, em contraste com os 188 misses do "Linha por Linha" (alta localidade).

Ficou provado que uma política de alocação flexível, como a Fully Associative, neutraliza essa ineficiência ao eliminar misses de conflito. Isso foi evidenciado pela queda de 321 misses (em Direct Mapping) para apenas 67 (em Fully Associative) no cenário de 16 blocos/8 words, igualando o desempenho do algoritmo otimizado.

Por fim, o experimento confirmou que, quando a capacidade da cache (ex: 2048B) excede o working set (matriz + strings), o impacto do algoritmo e da política de alocação é mitigado. O desempenho de ambos os programas converge para o número mínimo de 9 misses compulsórios. A simulação N-way Set Associative confirmou a teoria ao operar de forma idêntica ao Direct Mapping, pois foi executada com apenas 1 bloco no tamanho do conjunto.